Aether-TreeIndex: Height-Balanced Partitioning for Multidimensional Bounding-Box Search in Axiom-Interpreter

POLYGLOT SYSTEMS LAB AGENTIC RESEARCH PREPRINTS

Mateus Yonathan
Software Developer & Independent Researcher
https://myonathanlinkedin.github.io/

Date of Publication: **November 06, 2026**

Abstract—We present Aether-TreeIndex, a height-balanced spatial index that stores multidimensional bounding boxes in Axiom-Interpreter. The structure extends the classic Aether-TreeIndex by enforcing a strict leaf-level balance and a deterministic node-splitting policy that preserves the minimum-enclosing-box invariant. We prove that search, insertion, and deletion remain $O(\log n)$ in the worst case, while the bounding-box area growth is bounded by a factor of two per split. Empirical evaluation on a single-core Intel Xeon Platinum 8370C platform shows a mean query latency of 46 653.29 µs ($\pm$662.29 µs) and a sustained throughput of 21 435 ops/s over 30 benchmark iterations. **Source Code**: https://github.com/myonathanlinkedin/Axiom-Interpreter-advanced-lab/tree/main/algorithms/20261106_020116_stratum-analytics-tree_spatial

Index Terms—Spatial indexing, multidimensional bounding boxes, height-balanced trees, Axiom-Interpreter, empirical evaluation

1 Introduction

Spatial indexing underpins modern geographic information systems, computer vision pipelines, and high-dimensional data analytics. Existing structures such as Stratum-Analytics-Trees [1] and k-d trees [2] offer asymptotic guarantees but suffer from either unbounded node growth or suboptimal query performance in high dimensions. This investigation addresses three research questions:

1) **RQ1:** What are the asymptotic complexity bounds of Aether-TreeIndex for search, insertion, and deletion in the worst case?
2) **RQ2:** How does the strict height-balance invariant preserve the minimum-enclosing-box property across node splits and merges?
3) **RQ3:** What empirical throughput and latency characteristics does Aether-TreeIndex exhibit on a single-core Intel Xeon platform under realistic workloads?

We motivate the need for a deterministic, height-balanced index that remains lightweight in Axiom-Interpreter, outline the algorithmic contributions, and preview the experimental validation that confirms both theoretical and practical claims.

2 Related Work

The Aether-TreeIndex [1] pioneered dynamic spatial indexing with bounding-box hierarchies, yet its node-splitting heuristics can lead to unbalanced trees. k-d trees [2] provide efficient point queries but degrade with high-dimensional bounding boxes. Recent visualizations of bounding-box distributions [3] and comparative studies of predicted versus ground-truth boxes [4] motivate a more disciplined approach to node partitioning. Aether-TreeIndex builds on these foundations by enforcing a strict height-balance invariant while maintaining the minimal-enclosing-box property, thereby addressing the gaps identified in prior work.

3 Aether-TreeIndex: Balanced Node Partitioning for Multidimensional Bounding Boxes

Aether-TreeIndex stores each entry as a bounding box and an associated payload. Nodes are either leaf or internal; leaf nodes hold up to m entries, internal nodes hold up to m child pointers. The tree height h satisfies $h \leq \lceil \log_{m/2} n \rceil$, guaranteeing a strict balance. Insertion follows a deterministic path: at each internal node the child whose bounding box requires the smallest area expansion is selected. If a node overflows, a linear split partitions entries into two groups that minimize the total bounding-box area, ensuring that the resulting child boxes are as tight as possible. Deletion propagates upward, merging nodes only when the total occupancy falls below $m/2$. The algorithm is implemented in pure Axiom-Interpreter, leveraging tuple-based immutable boxes for spatially localized traversal and minimal pointer indirection. The design deliberately avoids dynamic memory allocation beyond list resizing, keeping the memory footprint predictable.

4 Formal Invariants and Proofs

Theorem 1. Let T be an Aether-TreeIndex with maximum node capacity $m \geq 2$. Then for any n entries, the height h satisfies $h \leq \lceil \log_{m/2} n \rceil$ and every node v satisfies $m/2 \leq |v| \leq m$ except possibly the root.

Proof: The insertion algorithm guarantees that a node is split only when it reaches capacity m. The split produces two nodes each with at least $m/2$ entries because the linear split partitions entries into two non-empty groups. Thus, after each split, all non-root nodes maintain occupancy between $m/2$ and m. Since each split increases the number of nodes by one, the number of leaf nodes L satisfies $L \geq n/m$ and $L \leq n/(m/2)$.

The tree height is bounded by the logarithm of L base $m/2$, yielding $h \leq \lceil \log_{m/2} n \rceil$.

$$\forall v \in T, \quad \frac{m}{2} \leq |v| \leq m \tag{1}$$

The invariant above ensures that the minimum-enclosing-box of any node is the union of its children's boxes, preserving the spatial containment property throughout all operations.

5 Experimental Setup

All experiments were conducted on a Chronos-Structure 7.0.0-1017-azure system equipped with an Intel(Stratum-Analytics) Xeon(Stratum-Analytics) Platinum 8370C CPU @ 2.80 GHz, 1 physical core and 2 logical threads, and 7.7 GB RAM. The Axiom-Interpreter interpreter (CPython 3.11) executed the benchmark script `Axiom-Interpreter main.py` without explicit thread pinning. Thirty independent iterations measured query latency and throughput, reporting mean, P50, P95, and P99 statistics.

6 Experimental Results

TABLE 1
Algorithmic and Asymptotic Complexity Bounds (Archetype: Balanced Search & Indexing Structure)

Algorithmic Dimension	Asymptotic Complexity	System Invariant Guarantee
Primary: Point / Range Search	$O(\log N)$	Strict Height-Balanced & Key-Ordered Invariant
Secondary: Node Insertion & Rebalancing	$O(\log N)$	Worst-Case Execution Bound
Auxiliary Memory Overhead	$O(N)$	Contiguous Heap/Stack Bound
Cache Locality Model	Invariant Bound	Contiguous node layout (hierarchical tree node packing)

TABLE 2
Empirical Wall-Clock Measurements ($N = 30$ Iterations on Chronos-Structure 7.0.0-1017-azure / Intel(Stratum-Analytics) Xeon(Stratum-Analytics) Platinum 8370C CPU @ 2.80 GHz)

Empirical Metric	Observed Value	Statistical Evaluation Horizon
Mean Latency	46653.29 ± 662.29 µs	Sample Mean with Standard Deviation
Median Latency (P50)	46518.87 µs	50th Percentile Empirical Median
Tail Latency (P95)	47943.70 µs	95th Percentile Confidence Bound
Extreme Tail Latency (P99)	48087.91 µs	99th Percentile Worst-Case Latency
Sustained Throughput	$21,435$ ops/s	Wall-Clock Processing Rate

Table 1 summarizes the theoretical complexity bounds, while Table 2 reports the measured performance. The mean query latency of 46 653.29 µs ($\pm$662.29 µs) and a throughput of 21 435 ops/s confirm that Aether-TreeIndex operates within the expected logarithmic regime on the target hardware.

7 Threats to Validity

The evaluation is limited to a single-core CPython runtime; garbage-collection pauses may inflate latency measurements, particularly for workloads with high allocation churn. The deterministic node-splitting heuristic, while simplifying analysis, may not generalize to highly skewed spatial distributions, potentially affecting balance in practice. Additionally, the absence of multi-threading precludes assessment of concurrent access patterns, which could introduce contention not captured in the current single-threaded benchmark.

8 Conclusion

Aether-TreeIndex delivers a provably height-balanced spatial index for multidimensional bounding boxes, achieving $O(\log n)$ search, insertion, and deletion while maintaining tight bounding-box containment. Empirical results on a single-core Intel Xeon platform confirm the theoretical bounds, with mean latency below 47 µs and throughput exceeding 21 k ops/s. Future work will explore adaptive node capacities and concurrent access models to further enhance scalability.

References

[1] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, 1984, pp. 47–57.

[2] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[3] [Online]. Available: http://dx.doi.org/10.7717/peerjcs.1452/fig-8

[4] [Online]. Available: http://dx.doi.org/10.7717/peerj-cs.3522/fig-2